

Haifa Capital (K) and K/L Series Construction Report

This report explains, in detail, how raw financial data from **Haifa Port Company (HPC)**, **Israel Ports Company (IPC)**, and **SIPG Haifa Bayport** are transformed into **three alternative monthly capital (K) series for each entity**, and how these are combined into **three Haifa-cluster K series**. It then explains how these capital series are merged with the **labor proxy** to construct **three monthly K/L series** for the Haifa cluster, plus a terminal-level K/L series for Haifa Legacy.

All series are expressed in **real 2019 NIS**, in **thousands** of NIS, and are built under three depreciation scenarios:

- **Low:** $\delta = 4\%$
- **Central:** $\delta = 6\%$
- **High:** $\delta = 8\%$

The architecture is deliberately parallel across HPC, IPC, and SIPG, so that the resulting K and K/L series can be used consistently in the econometric models.

1. Common Architecture and Conventions

1.1 Units and price base

Across all three entities (HPC, IPC, SIPG), the capital series are constructed in:

- **Real 2019 NIS** (constant prices),
- **Thousands of NIS** as the reporting unit.

To obtain these units, we:

1. Start from **nominal** PPE stocks and flows (net PPE from the balance sheet, capex and proceeds from the cash-flow statement, and depreciation & amortization from the income statement).
2. Convert foreign-currency values (SIPG) from **RMB to NIS** using an **annual average FX** series.
3. Deflate all NIS-denominated capital variables using the **same capital deflator** used in the HPC K pipeline (an OECD QNA index for non-residential investment, rebased to 2019 = 1).

After deflation, real values are divided by 1,000 to convert to **thousands** of NIS.

1.2 Three depreciation scenarios

For each entity we construct three capital paths by varying the geometric depreciation rate δ :

- Low: $\delta = 0.04$
- Central: $\delta = 0.06$

- High: $\delta = 0.08$

These scenarios are stored in small JSON meta files (e.g. `00_depreciation_choice.json`, `10_IPC_depreciation_choice.json`, `30_SIPG_Bayport_depreciation_choice.json`) that list the δ values and, where relevant, the year in which depreciation starts.

The **central** scenario is the default in most descriptive work, but the econometric code is written to allow sensitivity analyses using the low and high depreciation paths.

1.3 Annual PIM → Monthly interpolation

For each entity and each depreciation scenario we:

1. Build an **annual** capital series using a **Perpetual Inventory Method (PIM)** anchored to the first observed year in the financial statements.
2. Convert the annual PIM series to **monthly** capital using a **smooth interpolation** between annual **December** anchors.

The PIM recursion is:

$$K_t^{\text{PIM}} = (1 - \delta)^{\text{gap}} K_{t-1}^{\text{PIM}} + I_t - D_t,$$

where:

- K_t^{PIM} is the real capital stock at the end of year t ,
- δ is the depreciation rate (low/central/high),
- `gap` is the number of years between $t-1$ and t (usually 1),
- I_t is real investment (flows into capital),
- D_t is real disposals/retirements if explicitly available.

The result of this step is an annual table like:

- `year`
- `K_PIM_real_low`, `K_PIM_real_central`, `K_PIM_real_high` (real 2019 kNIS)
- `Dep_PIM_real_low`, `Dep_PIM_real_central`, `Dep_PIM_real_high`

The monthly builder scripts then:

1. Create a **monthly calendar** spanning the relevant period (e.g. 2018-01 to 2024-12).
2. Set **December** values of each year equal to the corresponding annual PIM stocks.
3. Use log-linear or linear interpolation between December anchors to fill in the months in between.

This yields monthly variables:

- `K_PIM_lin_low`, `K_PIM_lin_central`, `K_PIM_lin_high` (for IPC and SIPG), or
- `K_PathB_low`, `K_PathB_central`, `K_PathB_high` (for the lumpy HPC Path B).

1.4 Entity vs cluster series

Each entity (HPC, IPC, SIPG) has its own monthly K series. These are then combined into a **Haifa cluster** series by summing the entity-level capital stocks:

$$K^{\text{cluster}} = K^{\text{HPC}} + K^{\text{IPC}} + K^{\text{SIPG}}.$$

This is done separately for low, central, and high δ , so the Haifa cluster itself has **three K paths**.

Finally, these K series are merged with a **labor proxy** to construct **K/L** series.

2. Haifa Port Company (HPC) Capital Series

HPC is the legacy operator at Haifa. Its capital is built from detailed financial statement data using a two-track approach:

1. A **smooth PIM track** (Path A), and
2. A **lumpy Path B**, which incorporates project-level information on major investments and disposals.

Path B is used as the **preferred operator-level K series** for HPC.

2.1 Raw HPC financial data

Raw HPC data are collected from HPC's annual financial statements (2018–2024). The key components are:

- Balance sheet: net PPE (by class and total) at year end.
- PPE notes: roll-forward schedules showing opening cost, additions, disposals, transfers, and closing cost; accumulated depreciation; and net PPE.
- Cash-flow statement: lines for purchases of fixed assets, purchases of intangible assets, and proceeds from the sale of fixed assets.
- Income statement: total depreciation and amortization expense.

These are compiled into a hand-built TSV file (`haifa_financials_raw.tsv`) with standardized fields like:

- `ppe_net_nominal_thousands_nis_eoy`
- `depr_amort_total_thousands_nis_cf_adj`
- `purchase_of_fixed_assets_thousands_nis_cashflow`
- `purchase_of_intangible_assets_thousands_nis_cashflow`
- `proceeds_from_realization_fixed_assets_thousands_nis`

2.2 Step 1 – Nominal → real annual HPC financials

Script: `00_build_K_step1.py`

This script maps the raw HPC fields into a common PIM-ready structure:

- `ppe_net_nom` – total net PPE at year end.
- `additions_nom` – nominal gross capex (purchases of fixed and intangible assets, adjusted for classification choices).
- `depr_nom` – total depreciation & amortization expense.
- `disposals_nom` – proceeds from sale of fixed assets (or 0 where not available).

The script then:

1. Merges this annual panel with the **OECD capital deflator**, rebased so that 2019 = 1.
2. Computes real counterparts:
3. `K_book_real = ppe_net_nom / deflator`
4. `I_real = additions_nom / deflator`
5. `depr_real = depr_nom / deflator`
6. `disposals_real = disposals_nom / deflator`
7. Writes out a **Step 1** table with both nominal and real variables.
8. Creates a **depreciation meta JSON** listing low/central/high δ values.

2.3 Step 2 – Annual PIM for HPC

Script: `01_build_K_annual_PIM.py`

Using the Step 1 real variables and the δ meta file, this script:

1. Anchors the PIM at the first observed year:
2. `K_PIM_real_central(first year) = K_book_real(first year)`
3. Low and high scenarios are anchored in the same way.
4. Applies the PIM recursion to generate:
5. `K_PIM_real_low`, `K_PIM_real_central`, `K_PIM_real_high`
6. `Dep_PIM_real_low`, `Dep_PIM_real_central`, `Dep_PIM_real_high`
7. Computes QA gaps versus the accounting book values:
8. `gap_book_minus_PIM_central = K_book_real - K_PIM_real_central`, etc.

The output is a clean annual PIM table for HPC under the three depreciation scenarios.

2.4 Step 3 – Monthly PIM-lin (Path A)

Script: `02_build_K_monthly_PIM_linear.py`

This script creates a **smooth** monthly K series for HPC by:

1. Creating a monthly date range spanning the years in the annual PIM table.
2. Setting the monthly K values in **December** equal to the annual PIM stocks:
3. `K_PIM_lin_central(Dec Y) = K_PIM_real_central(Y)`, etc.
4. Interpolating monthly values between December anchors.

The result is a monthly table with:

- `K_PIM_lin_low`, `K_PIM_lin_central`, `K_PIM_lin_high`.

This is Path A. It is internally consistent but smooths away lumpy investments.

2.5 Step 4 – Lumpy Path B for HPC

Script: `03_build_K_monthly_PathB_lumpy.py`

To better reflect the timing of major capital projects, we build a lumpy **Path B** using project-level data in `Haifa_big_projects.csv`.

Each project entry includes:

- A description (e.g. STS crane purchase, yard expansion),
- Total cost (nominal),
- An approximate commissioning date,
- An assumed service life.

The script:

1. Deflates each project's cost into **real 2019 NIS**.
2. For each project and each δ scenario, constructs a **project-level K path**:
3. Zero before commissioning,
4. Full cost at commissioning,
5. Geometric decay at rate δ thereafter.
6. Sums project-level K across projects to obtain `K_big_low`, `K_big_central`, `K_big_high`.
7. At each year's December, compares `K_big_*` to the PIM stocks and infers a **background capital stock**:
8. `K_bg_scenario(Y) = K_PIM_real_scenario(Y) - K_big_scenario(Y)`.
9. Interpolates `K_bg_scenario` to monthly frequency.
10. Defines **Path B** as:
11. `K_PathB_scenario(t) = K_big_scenario(t) + K_bg_scenario(t)`.

This yields:

- `K_PathB_low`, `K_PathB_central`, `K_PathB_high` for HPC, at monthly frequency.

Path B is then treated as the **preferred operator-level K series** for HPC and is the one used in the Haifa cluster construction and K/L calculations.

3. Israel Ports Company (IPC) Capital Series

IPC is the landlord / infrastructure owner. For Haifa, IPC owns breakwaters, berths, yards, buildings, and other infrastructure. Its capital is built in a way parallel to HPC, but without a lumpy Path B.

3.1 Raw IPC financial data

Two main raw tables are prepared for the Haifa cluster:

1. **Table 1 – PPE position and movement by asset class**

2. For each year and asset class (marine structures, buildings, land, work-in-progress, etc.), includes:

- Opening cost, additions, transfers, disposals,
- Capitalised interest,
- Closing cost,
- Opening and closing accumulated depreciation,
- Net PPE.

3. **Table 2 – Cash-flow style summary**

4. For the Haifa cluster, includes:

- Depreciation & amortization,
- Purchase of PPE,
- Capitalised interest on PPE,
- Proceeds from sale of PPE.

5. From these, an `approx_gross_ppe_investment` variable is constructed.

3.2 Step 10 – IPC nominal → real annual series

Script: `10_build_IPC_step1_real.py`

This script constructs an annual IPC panel with the same core structure as HPC:

- `ppe_net_nom` – from the total net PPE in Table 1.
- `additions_nom` – from `approx_gross_ppe_investment` in Table 2.
- `depr_nom` – total depreciation & amortization.
- `disposals_nom` – proceeds from sale of PPE (or 0 if treated differently).

It then:

1. Merges with the same **OECD capital deflator**.
2. Computes:
3. `K_book_real`, `I_real`, `depr_real`, `disposals_real`.
4. Writes an IPC Step 1 table and a δ meta JSON.

3.3 Step 11 – Annual PIM for IPC

Script: `11_build_K_annual_IPC_PIM.py`

This script is effectively a copy of the HPC PIM script, adapted to IPC field names. It:

1. Anchors the PIM in the first observed year, using `K_book_real`.

2. Applies the PIM recursion for low/central/high δ .
3. Produces annual PIM stocks and depreciation for IPC.

3.4 Step 12 – Monthly IPC PIM-lin

Script: `12_build_K_monthly_IPC_PIM_linear.py`

This script mirrors HPC's PIM-linear builder:

1. Constructs a monthly date range.
2. Places annual PIM stocks at each December.
3. Interpolates monthly K values between December anchors.

The resulting monthly series:

• `K_PIM_lin_low`, `K_PIM_lin_central`, `K_PIM_lin_high`,

are interpreted as IPC's **Haifa-cluster infrastructure capital** under the three depreciation scenarios.

4. SIPG Haifa Bayport Capital Series

SIPG operates the new Haifa Bayport container terminal. Its capital is not reported in Israeli NIS accounts, so we construct the series from a combination of **CIP data in RMB** and an FX conversion step.

4.1 Raw SIPG CIP data (in RMB)

The key raw file is `sipg_haifa_bayport_CIP_RMB.tsv`. It has one row per **project group × year**, with columns:

- `year`
- `project_group` (e.g. "Bayport main terminal", "Bayport ancillary systems")
- `budget_rmb`
- `cip_opening_rmb`
- `cip_additions_rmb`
- `cip_transfers_to_ppe_rmb`
- `cip_other_decreases_rmb`
- `cip_closing_rmb`
- `cip_cumul_capitalized_interest_rmb`
- `cip_current_capitalized_interest_rmb`

The file encodes the evolution of **construction-in-progress (CIP)** related to the Haifa Bayport, in RMB.

4.2 FX conversion: daily CNY/ILS → annual averages

SIPG's CIP is denominated in RMB, so we need an **RMB→NIS** conversion.

We start from a daily CNY/ILS data file, `CNY_ILS_Historical_Data.csv`, which contains daily rates with a `Date` column and a numeric rate column (typically `Price`, the ILS per 1 CNY).

The SIPG Step 1 script automatically:

1. Loads the daily CNY/ILS series.
2. Parses the `Date` column as a date and extracts the `year`.
3. Groups by `year` and computes the **annual average** rate.
4. Writes a small TSV file `fx_rmb_nis.tsv` with columns:
5. `year`
6. `fx_avg_nis_per_rmb`.

This annual FX series is then used for all RMB → NIS conversions.

4.3 Step 30 – Annual SIPG flows and stocks in real 2019 NIS

Script: `30_build_SIPG_Bayport_step1_real.py`

This script converts the CIP data to a consolidated annual panel of Bayport investment and capital in **real 2019 NIS (thousands)**.

4.3.1 Clean and aggregate CIP data

The script:

1. Reads `sipg_haifa_bayport_CIP_RMB.tsv`.
2. Ensures numeric columns are coerced to floats, with non-numeric strings turned into `NaN`.
3. Constructs annual CIP flows in RMB:
4. `I_RMB = cip_additions_rmb + cip_current_capitalized_interest_rmb`
5. `T_RMB = cip_transfers_to_ppe_rmb` (transfers from CIP to PPE)
6. `E_RMB = cip_other_decreases_rmb`
7. Aggregates across project groups to the **annual Haifa Bayport level**, summing:
8. `budget_rmb`, `cip_opening_rmb`, `cip_closing_rmb`, `I_RMB`, `T_RMB`, `E_RMB`.

It then performs a consistency check:

- CIP identity: `closing ≈ opening + I - T - E`.

Any discrepancies (notably in 2021) are reported as a warning but not forced to zero, reflecting minor data imperfections in the raw CIP extraction.

4.3.2 Build completed and gross capital in RMB

The script constructs two key cumulative series:

- `K_completed_RMB = cumulative sum of T_RMB` (total completed Bayport capital placed into service).

- $K_gross_RMB = K_completed_RMB + cip_closing_rmb$ (completed capital plus remaining CIP).

4.3.3 Convert from RMB to real 2019 NIS (thousands)

Next, the script:

1. Merges the annual CIP panel with the **annual FX table** ($fx_avg_nis_per_rmb$).
2. Converts to NIS:
3. $I_NIS = I_RMB * fx_avg_nis_per_rmb$
4. $T_NIS = T_RMB * fx_avg_nis_per_rmb$
5. $K_completed_NIS = K_completed_RMB * fx_avg_nis_per_rmb$
6. $K_gross_NIS = K_gross_RMB * fx_avg_nis_per_rmb$
7. Merges with the **capital deflator** and obtains real 2019 NIS by dividing each NIS series by the deflator.
8. Converts to thousands:
9. I_real_kNIS , T_real_kNIS ,
10. $K_completed_real_kNIS$, $K_gross_real_kNIS$.

The output of Step 30 is an annual panel with one row per year and the key series needed for the PIM. It also writes a meta JSON specifying:

- The available depreciation scenarios (low/central/high), and
- The **I_productive_definition**, which is set to "transfers_only".

4.4 Step 31 – Annual PIM for SIPG

Script: `31_build_K_annual_SIPG_Bayport_PIM.py`

This script builds an **annual PIM K series** for SIPG Haifa Bayport.

Key design choices:

- **Productive investment** I_t is defined as **transfers from CIP to PPE only**, i.e. $I_t = T_real_kNIS$.
- Depreciation begins only once Bayport is **operational**, at $year_dep_start = 2021$.

The PIM recursion is applied using $I_t = T_real_kNIS$ and δ in {0.04, 0.06, 0.08}. Before 2021, the PIM capital stock is kept at zero and no depreciation is applied (reflecting the construction phase). From 2021 onward, the usual PIM logic applies.

The script outputs an annual table with:

- $K_PIM_real_low$, $K_PIM_real_central$, $K_PIM_real_high$,
- $Dep_PIM_real_low$, $Dep_PIM_real_central$, $Dep_PIM_real_high$,

in real 2019 NIS (thousands), plus meta information about the PIM and depreciation start year.

4.5 Step 32 – Monthly SIPG PIM-lin

Script: `32_build_K_monthly_Haifa_SIPG_PIM_linear.py`

This script translates the annual SIPG PIM table into monthly K series.

1. It constructs a monthly date range from January 2018 through December 2024.
2. For each δ scenario (low/central/high), it:
3. Sets the December value of each year equal to that year's PIM stock.
4. Sets all months **before the first positive PIM stock (2021–12)** to zero.
5. Interpolates monthly K values between December anchors.

The result is a file `32_K_B_monthly_Haifa_SIPG_PIM_lin.tsv` with:

- `K_PIM_lin_low`, `K_PIM_lin_central`, `K_PIM_lin_high` for SIPG,
- One row per month from 2018–01 to 2024–12.

5. Combining HPC, IPC, and SIPG into Haifa Cluster K

Once HPC, IPC, and SIPG monthly K series exist, they are combined into a single **Haifa cluster** panel that tracks capital for all three entities under each δ scenario.

5.1 Existing HPC + IPC cluster file (Step 20)

Script: `20_build_Haifa_cluster_K_merge.py`

This pre-existing script constructs `20_K_cluster_Haifa_HPC_IPC_monthly.tsv`, which:

1. Loads the HPC Path B monthly series.
2. Loads IPC's monthly PIM-lin series.
3. Aligns them by `(port, month, year)`.
4. For each month:
5. Defines `K_HPC_*` as the HPC Path B K,
6. Defines `K_IPC_*` as the IPC PIM-lin K,
7. Constructs a cluster K (without SIPG): `K_cluster_* = K_HPC_* + K_IPC_*`.

This file is in long format, with:

- Rows for HPC,
- Rows for IPC,
- Rows for the cluster `"Haifa cluster (HPC + IPC)"`.

For each month, the HPC rows carry the decomposition columns `K_HPC_*`, `K_IPC_*`, `K_cluster_*`.

5.2 Adding SIPG and constructing the new cluster with three entities

Script: `21_build_Haifa_cluster_K_merge_with_SIPG.py`

This script creates a new cluster panel, `21_K_cluster_Haifa_HPC_IPC_SIPG_monthly.tsv`, that includes SIPG.

The main steps:

1. **Load the HPC+IPC base** from `20_K_cluster_Haifa_HPC_IPC_monthly.tsv`:
2. Filter to rows where `company == "Haifa Port Company (legacy)"`.
3. These rows already contain, for each month and scenario:
 - `K_HPC_low`, `K_HPC_central`, `K_HPC_high`,
 - `K_IPC_low`, `K_IPC_central`, `K_IPC_high`.
4. **Load the SIPG monthly series** from `32_K_B_monthly_Haifa_SIPG_PIM_lin.tsv`:
5. Filter to `company == "SIPG Haifa Bayport"`.
6. Rename `K_PIM_lin_*` to `K_SIPG_low`, `K_SIPG_central`, `K_SIPG_high`.
7. **Merge HPC+IPC base with SIPG by** `(port, year, month)` using an inner join, restricting to the overlapping months.
8. **Construct new cluster K for each scenario:**
9. `K_cluster_low = K_HPC_low + K_IPC_low + K_SIPG_low`,
10. `K_cluster_central = K_HPC_central + K_IPC_central + K_SIPG_central`,
11. `K_cluster_high = K_HPC_high + K_IPC_high + K_SIPG_high`.
12. **Create a long panel with four company blocks** for each month:
13. **HPC rows:**
 - `company = "Haifa Port Company (legacy)"`,
 - `K_low = K_HPC_low`, `K_central = K_HPC_central`, `K_high = K_HPC_high`.
14. **IPC rows:**
 - `company = "Israel Ports Company (Haifa cluster)"`,
 - `K_low = K_IPC_low`, `K_central = K_IPC_central`, `K_high = K_IPC_high`.
15. **SIPG rows:**
 - `company = "SIPG Haifa Bayport"`,
 - `K_low = K_SIPG_low`, `K_central = K_SIPG_central`, `K_high = K_SIPG_high`.

16. Cluster rows:

- `company = "Haifa cluster (HPC + IPC + SIPG)"`,
- `K_low = K_cluster_low`, `K_central = K_cluster_central`, `K_high = K_cluster_high`.

Each row also carries the full decomposition columns

- `K_HPC_*`, `K_IPC_*`, `K_SIPG_*`, `K_cluster_*`,

allowing full decomposition of cluster capital at any date.

The final file `21_K_cluster_Haifa_HPC_IPC_SIPG_monthly.tsv` thus contains **three Haifa K series** (low/central/high δ) at the **cluster** level, fully consistent with the entity-level series.

6. Constructing Monthly K/L Series

The final step is to combine the capital series with a **labor proxy** to construct K/L series. This is done by `Build_KL_Panel.py`.

6.1 Labor proxy data

The labor proxy comes from `L_Proxy.tsv`, which contains a **monthly panel of labor inputs** by port and terminal.

Key columns include:

- `port` (e.g. "Haifa"),
- `terminal` (e.g. "Haifa-Legacy"),
- `year`,
- `month`,
- `L_hours_i_m` – a labor input measure (e.g. total labor hours at terminal i in month m).

This file is produced elsewhere in the labor pipeline, and is treated here as an input.

6.2 Haifa Legacy terminal-level K/L

First, we construct a **terminal-level** K/L series for the legacy Haifa operator.

6.2.1 Matching HPC K with Haifa-Legacy labor

We:

1. Load `21_K_cluster_Haifa_HPC_IPC_SIPG_monthly.tsv` and select rows with:
2. `company == "Haifa Port Company (legacy)"`.
3. Use `K_central` as HPC's operator-level capital (Path B, central δ).

4. Load `L_Proxy.tsv` and filter to:
5. `port == "Haifa"`,
6. `terminal == "Haifa-Legacy"`.
7. Merge the two tables on `(port, year, month)`.

6.2.2 Computing K/L for Haifa Legacy

In the merged panel we:

- Rename `L_hours_i_m` to `L` and `K_central` to `K`.
- Drop any rows where `K <= 0` or `L <= 0`.
- Compute:
 - `KL = K / L`,
 - `log_KL = ln(KL)`.
- Construct a `month_index = year * 12 + month` (aligning with other panels).
- Add metadata:
 - `series_id = "Haifa_Legacy_KL"`,
 - `level = "terminal"`,
 - `freq = "M"`.

This yields a terminal-level K/L series that can be used in regressions that focus on the legacy operator's productivity relative to its own capital stock.

6.3 Haifa cluster K/L – three depreciation scenarios

Next, we build **three port-level K/L series** using Haifa's **cluster capital** and the sum of labor across all Haifa terminals.

6.3.1 Cluster K for each depreciation scenario

From `21_K_cluster_Haifa_HPC_IPC_SIPG_monthly.tsv` we take the rows where:

- `company == "Haifa cluster (HPC + IPC + SIPG)"`.

These rows contain, for each month:

- `K_low`, `K_central`, `K_high` – cluster capital for low/central/high δ .

6.3.2 Cluster labor – sum over all Haifa terminals

From `L_Proxy.tsv` we compute a **port-level labor series** by summing over terminals:

- Group by `(port, year, month)` and sum `L_hours_i_m` across all Haifa terminals.
- Keep rows where `port == "Haifa"`.

This yields a panel `L_haifa_port` with:

- `port`, `year`, `month`, `L` (total Haifa labor hours for that month).

6.3.3 Constructing K/L for each scenario

We then merge the cluster capital and port-level labor by `(port, year, month)`, obtaining a table with:

- `L`,
- `K_low`, `K_central`, `K_high`.

For each depreciation scenario, we create a separate K/L series:

1. **Low depreciation** - `Haifa_port_KL_cluster_low`
2. Set `K = K_low`.
3. Drop rows with `K <= 0` or `L <= 0`.
4. Compute:
 - `KL = K / L`,
 - `log_KL = ln(KL)`.
5. Set:
 - `series_id = "Haifa_port_KL_cluster_low"`,
 - `level = "port"`,
 - `freq = "M"`,
 - `terminal = NaN` (port-level series).
6. **Central depreciation** - `Haifa_port_KL_cluster_central`
7. Identical steps but with `K = K_central`.
8. **High depreciation** - `Haifa_port_KL_cluster_high`
9. Identical steps but with `K = K_high`.

In each case we also construct `month_index` as `year * 12 + month` for alignment.

The outputs of these three series are concatenated into a single K/L panel that includes:

- The Haifa Legacy terminal-level K/L series, and
- The three Haifa cluster K/L series under low, central, and high δ .

6.4 Final K/L panel

`Build_KL_Panel.py` writes the combined K/L panel to:

- `Data/KL/KL_Panel_monthly.tsv`.

Each row includes:

- `series_id` – identifies which series (Haifa_Legacy_KL, Haifa_port_KL_cluster_low, etc.).
- `level` – "terminal" or "port".
- `freq` – "M" for monthly.
- `port` – "Haifa".
- `terminal` – specific terminal name for terminal-level series, or `NaN` for port-level.
- `year`, `month`, `month_index`.
- `K` – capital stock (real 2019 kNIS) relevant to this series.
- `L` – labor proxy (total labor hours in that terminal or port).
- `KL` – capital per labor.
- `log_KL` – log capital per labor.

These K/L series are now **consistent** with the K construction for HPC, IPC, and SIPG, incorporate the three depreciation scenarios, and are ready to be merged into the econometric analysis (e.g. Models 1B and 2) as either outcomes or mediators.

7. Summary

1. **HPC:** Capital is built from net PPE, capex, and disposals using a Perpetual Inventory Method and a lumpy Path B that incorporates project-level information. The result is three operator-level monthly K series (low/central/high δ).
2. **IPC:** Capital is built from Haifa cluster PPE and investment tables, mapped into the same Step 1 → PIM → monthly PIM-lin architecture. The result is three monthly K series for the landlord's infrastructure.
3. **SIPG Haifa Bayport:** Capital is built from RMB CIP data, converted to NIS via annual average FX and deflated using the same capital deflator. Transfers from CIP to PPE serve as the productive investment flows in a PIM with depreciation starting at commissioning (2021). The result is three monthly PIM-lin K series for SIPG.
4. **Haifa cluster:** The three entity-level series are combined to produce three Haifa cluster K paths (`K_low`, `K_central`, `K_high`) in `21_K_cluster_Haifa_HPC_IPC_SIPG_monthly.tsv`, with full decomposition by owner/operator.
5. **K/L series:** The Haifa K series (cluster-level and HPC operator-level) are merged with a terminal×month labor proxy to produce:
6. One Haifa Legacy terminal-level K/L series (`Haifa_Legacy_KL`), and
7. Three Haifa port-level K/L series (`Haifa_port_KL_cluster_low`, `..._central`, `..._high`).

This unified pipeline provides a transparent, internally consistent set of capital and K/L series for Haifa that can be used for event-study and difference-in-differences analyses of the port reforms, competition, and privatization episodes.